

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionState](#)

frontend/src/config/api.service.js

```
1.  /**
2.   * @file Configuración del servicio de API con Axios.
3.   * @description Crea una instancia de Axios con configuración base e
4.   * @requires axios
5.   */
6.  import axios from 'axios';
7.
8.  /**
9.   * @constant {string} API_URL
10.   * @description URL base de la API del backend.
11.   * Lee de la variable de entorno REACT_APP_API_URL.
12.   * Fallback a http://localhost:4000 solo para desarrollo local.
13.   * En producción, REACT_APP_API_URL DEBE estar configurada.
14.   */
15.  const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:4000';
16.
17.  /**
18.   * @constant {AxiosInstance} api
19.   * @description Instancia de Axios configurada para las peticiones a
20.   */
21.  const api = axios.create({
22.    baseURL: API_URL,
23.    timeout: 10000,
24.    headers: {
25.      'Content-Type': 'application/json'
26.    }
27.  });
28.
29.  /**
30.   * @function request-interceptor
31.   * @description Interceptor de peticiones de Axios que añade el token
32.   */
33.  // Interceptor para añadir token automáticamente
34.  api.interceptors.request.use(
35.    (config) => {
36.      // Obtener token desde Zustand persist storage
37.      const authStorage = localStorage.getItem('auth-storage');
38.      if (authStorage) {
39.        try {
40.          const { state } = JSON.parse(authStorage);
41.          const token = state?.token;
42.          if (token && token !== '') {
43.            config.headers.Authorization = `Bearer ${token}`;
44.          }
45.        } catch (error) {
46.          console.error('Error parsing auth storage:', error);
47.        }
48.      }
49.      return config;
50.    },
51.    (error) => {
52.      return Promise.reject(error);
```

```
53.     }
54.   );
55.
56.   /**
57.    * @function response-interceptor
58.    * @description Interceptor de respuestas de Axios que maneja errores
59.    */
60.   // Interceptor para manejar errores
61.   api.interceptors.response.use(
62.     (response) => response,
63.     async (error) => {
64.       if (error.response?.status === 401) {
65.         // Limpiar el storage de Zustand
66.         localStorage.removeItem('auth-storage');
67.         // Redirigir al login
68.         window.location.href = '/login';
69.       }
70.
71.       const formattedError = {
72.         message: error.response?.data?.message || error.message,
73.         status: error.response?.status,
74.         data: error.response?.data
75.       };
76.
77.       return Promise.reject(formattedError);
78.     }
79.   );
80.
81.   export default api;
```

Documentation generated by [JSDoc 4.0.5](#) on Thu Dec 11 2025 18:33:28 GMT+0000
(Coordinated Universal Time) using the [docdash](#) theme.